PROJECT 4 (MATH214)

MUHAMMET ALI AYVAZ

210102002078

In [8]:
```
import numpy as np
from scipy.linalg import inv #for task1..
from scipy.linalg import lu, solve #for task3..
from scipy.linalg import cholesky #for task4..
```

1) The Gauss-Seidel method algorithm was implemented, but it failed to solve this matrix. After extensive research, as indicated in the references section at the end of the report [1], it was found on Wikipedia that the reason for this issue is related to the concept of a Diagonally Dominant matrix. The matrix used in Task 1 with the Gauss-Seidel method is not suitable because it lacks a dominant diagonal, as confirmed during investigations. Additionally, it was observed that the library also does not support the Gauss-Seidel method.

In [9]:
```
# Coefficient matrix
A = np.array([
    [2, -1, 2, 0],
    [2, -2, -1, 1],
    [2, 2, 1, 0],
    [2, -1, 2, 2]
])

# Constant term vector
b = np.array([8, 5, 4, 10])

# Solution using the solve function in the NumPy library
x_np = np.linalg.solve(A, b)

# Solution by taking the inverse of the matrix using the inv function in the NumPy
x_inv = np.dot(np.linalg.inv(A), b)

# Print the results
print("solve function :", x_np)
print("inv function :", x_inv)
```

```
solve function : [ 2.  -0.8  1.6  1. ]
inv function : [ 2.  -0.8  1.6  1. ]
```

2) Cramer's method is a technique to solve a system of linear equations by systematically replacing each column of the coefficient matrix and calculating determinants. Here's a similar explanation with the same meaning:

The coefficient matrix (A) and the result vector (B) are given. The determinant of (A) is computed using `np.linalg.det(A)` and stored in the variable `determinant`. According to Cramer's rule, the solutions for (x_1), (x_2), and (x_3) are found as follows:

- (x_1): `np.linalg.det(np.column_stack((B, A[:, 1], A[:, 2]))) / determinant`
- (x_2): `np.linalg.det(np.column_stack((A[:, 0], B, A[:, 2]))) / determinant`
- (x_3): `np.linalg.det(np.column_stack((A[:, 0], A[:, 1], B))) / determinant`

These equations isolate each unknown using determinants. Finally, the solution is displayed with the statement:
`"The solution to the system of equations using Cramer's method is: x1 = {}, x2 = {}, x3 = {}".format(x1, x2, x3)`

In [10]:
```
# Given coefficient matrix A and result vector B
A = np.array([[2, 3, -1], [1, -2, 2], [-1, 4, 1]])
B = np.array([4, 6, 5])

# Calculate the determinant
determinant = np.linalg.det(A)

# Finding the x(n)...
x1 = np.linalg.det(np.column_stack((B, A[:, 1], A[:, 2]))) / determinant
x2 = np.linalg.det(np.column_stack((A[:, 0], B, A[:, 2]))) / determinant
x3 = np.linalg.det(np.column_stack((A[:, 0], A[:, 1], B))) / determinant

# showing the results
print("The solution to the system of equations is (with Cramer's method): x1 = {},
```

```
The solution to the system of equations is (with Cramer's method): x1 = 2.00000000
00000004, x2 = 1.0, x3 = 3.000000000000001
```

3) LU factorization is used to solve the system of linear equations. Here's a similar explanation maintaining the same core idea:

The coefficient matrix (A) and the result vector (B) are given. The LU factorization is performed using the `lu(A)` function, which decomposes (A) into a lower triangular matrix (L), an upper triangular matrix (U), and a permutation matrix (P). These are stored in the variables (P), (L), and (U), respectively.

First, the equation (Ly = B) is solved using the lower triangular matrix (L). This is achieved with `solve(L, B)`, and the result is stored in the variable (y). Next, the equation (Ux = y) is solved using the upper triangular matrix (U). This is performed with `solve(U, y)`, and the solution is stored in the variable (x).

Finally, the solution is displayed using the statement:
`"Solution to the system of equations (using LU): x1 = {}, x2 = {}, x3 = {}".format(x[0], x[1], x[2])`

In [11]:
```
# Given coefficient matrix A and result vector B
A = np.array([[2, -1, 1], [3, 3, 9], [3, 3, 5]])
B = np.array([-1, 0, 4])

# LU factorization
P, L, U = lu(A)

# Solve Ly = B
y = solve(L, B)
```

```
# Solve Ux = y
x = solve(U, y)

# Print the results
print("Solution to the system of equations(with LU): x1 = {} , x2 = {}, x3 = {}".fo
```

```
Solution to the system of equations(with LU): x1 = 1.5555555555555556 , x2 = 1.861
111111111111, x3 = -1.25
```

4) Cholesky factorization is a technique that decomposes a symmetric positive definite matrix into the product of a lower triangular matrix and its transpose, acting as a square root for such matrices. It is especially effective in numerical computations, and this code compares the results of NumPy and SciPy implementations to verify their consistency.

The given matrix is defined and stored in the variable (A). Cholesky factorization is performed using the `np.linalg.cholesky(A)` function from the NumPy library, and the result is stored in the variable `c_np`. Similarly, Cholesky factorization is performed using SciPy's `cholesky(A, lower=True)` function, and the result is assigned to the variable `c_scipy`.

The function `np.allclose(c_np, c_scipy)` is used to compare the two results, returning a boolean value stored in the variable `flag`. The outcome is then printed:

- If `flag` is `True`, it displays: "NumPy and SciPy Cholesky functions produce the same result."
- If `flag` is `False`, it displays: "There is a difference between NumPy and SciPy Cholesky functions."

In [12]:

```
# writing the given matrix
A = np.array([[4, 2, 2], [2, 6, 2], [2, 2, 5]])

# in numpy
c_np = np.linalg.cholesky(A)

# in scipy
c_scipy = cholesky(A, lower=True)

# Comparison flag
flag = np.allclose(c_np, c_scipy)

# Print the results
print("NumPy Cholesky Factorization:\n", c_np)
print("\nSciPy Cholesky Factorization:\n", c_scipy)

if flag:
    print("\nnumpy and scipy cholesky functions produce the same result.")
else:
    print("\nThere is a difference between numpy and scipy cholesky functions.")
```

```
NumPy Cholesky Factorization:
 [[2.         0.         0.        ]
 [1.         2.23606798 0.        ]
 [1.         0.4472136  1.94935887]]

SciPy Cholesky Factorization:
 [[2.         0.         0.        ]
 [1.         2.23606798 0.        ]
 [1.         0.4472136  1.94935887]]

numpy and scipy cholesky functions produce the same result.
```

5) The code computes the eigenvalues, eigenvectors, and spectral radii for two distinct matrices.

First, a matrix (a1) is defined. Its eigenvalues ((e_val1)) and eigenvectors ((e_v1)) are calculated using the `np.linalg.eig(a1)` function. The spectral radius ((s_rad1)) is determined as the largest absolute value of the eigenvalues.

Next, another matrix (a2) is defined. Similarly, its eigenvalues ((e_val2)) and eigenvectors ((e_v2)) are calculated using `np.linalg.eig(a2)`. The spectral radius ((s_rad2)) is obtained as the maximum absolute value of the eigenvalues.

The computed results for both matrices are then displayed, including their eigenvalues, eigenvectors, and spectral radii.

In [13]:

```python
# matrix A1
a1 = np.array([[1, 2], [-1, 2]])

# eigenvalues and eigenvectors
e_val1, e_v1 = np.linalg.eig(a1)

# Spectral radius
s_rad1 = max(abs(e_val1))

# showing the results
print("matrix a1:")
print("eigen values:", e_val1)
print("eigen vectors:\n", e_v1)
print("spectral radius:", s_rad1)


# Matrix A2
a2 = np.array([[-1, 2, 0], [0, 3, 4], [0, 0, 7]])

# Eigenvalues and eigenvectors
e_val2, e_v2 = np.linalg.eig(a2)

# Spectral radius
s_rad2 = max(abs(e_val2))

# Print the results
print("\nmatrix a2:")
print("eigen values:", e_val2)
print("eigen vectors:\n", e_v2)
print("spectral radius:", s_rad2)
```

```
matrix a1:
eigen values: [1.5+1.32287566j 1.5-1.32287566j]
eigen vectors:
 [[0.81649658+0.j         0.81649658-0.j        ]
 [0.20412415+0.54006172j 0.20412415-0.54006172j]]
spectral radius: 2.0

matrix a2:
eigen values: [-1.  3.  7.]
eigen vectors:
 [[1.         0.4472136  0.17407766]
 [0.         0.89442719 0.69631062]
 [0.         0.         0.69631062]]
spectral radius: 7.0
```

REFERENCES

1. https://en.wikipedia.org/wiki/Diagonally_dominant_matrix
2. https://www.youtube.com/watch?v=Gt19Tz0PGYU
3. https://www.youtube.com/watch?v=eAsJ7XUsdxo
4. https://www.youtube.com/watch?v=4SWMzENcgSE
5. https://numpy.org/doc/stable/reference/generated/numpy.linalg.eig.html

In [ ]:

In [ ]: